# Lab Final

Irene

2024-05-30

```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
library(glmnet)
```

```
## Warning: package 'glmnet' was built under R version 4.5.2
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 5.2
```

```
set.seed(1234)
data <- read.csv("Competition_data copy.csv")

# Remove ID and zip code
data <- subset(data, select = -c(HHKEY, ZIP_CODE))
# Change to factors
data$CC_CARD <- as.factor(data$CC_CARD)
data$WEB <- as.factor(data$WEB)
data$RESP <- as.factor(data$RESP)
data$CLUSTYPE <- as.factor(data$CLUSTYPE)

# Convert VALPHON to a single dummy variable
data$VALPHON <- ifelse(data$VALPHON == "Y", 1, 0)

# Dummy variables for CLUSTYPE
dummy_matrix <- model.matrix(~ CLUSTYPE - 1, data = data)
# Combine dummy variables with the original dataframe
data <- cbind(data, dummy_matrix)

# Remove the original factor column if needed
data$CLUSTYPE <- NULL

# Split data into training and testing sets
#train_indices <- createDataPartition(data$RESP, p = 0.8, list = FALSE)
#train_data <- data[train_indices, ]
```

```
#test_data <- data[-train_indices, ]

# Split into training (60%) and remaining (40%)
train_index <- createDataPartition(data$RESP, p = 0.60, list = FALSE)

train_data <- data[train_index, ]
remaining_data <- data[-train_index, ]

# Split remaining data into validation (20%) and testing (20%)
test_index <- createDataPartition(remaining_data$RESP, p = 0.50, list = FALSE)

test_data <- remaining_data[test_index, ]
validation_data <- remaining_data[-test_index, ]

counts <- table(data$RESP)
prop.table(counts)
```

```
##
##         0         1
## 0.8337076 0.1662924
```

Scale data

```
# scale using FITTED scaler to train_data
preProc <- preProcess(subset(train_data, select = -c(RESP, CC_CARD, WEB, CLUSTYPE1, CLUSTYPE2, CLUSTYPE3

# Transform data
train_scaled <- predict(preProc, train_data)
val_scaled <- predict(preProc, validation_data)
test_scaled <- predict(preProc, test_data)
```

Lasso regression to find optimal lambda and then select variables

```
set.seed(1234)
# Versions of train and test
X_train <- subset(train_scaled, select = -RESP)
X_test <- subset(test_scaled, select = -RESP)
X_val <- subset(val_scaled, select = -RESP)

X_train$WEB <- as.numeric(X_train$WEB)
X_train$VALPHON <- as.numeric(X_train$VALPHON)
X_train$CC_CARD <- as.numeric(X_train$CC_CARD)

y_train <- train_scaled$RESP
y_test <- test_scaled$RESP
y_val <- val_scaled$RESP

# Lasso to find predictors
# Fit Lasso model using cross-validation to find the optimal lambda
cv_fit <- cv.glmnet(as.matrix(X_train), y_train, alpha = 1, standardize = FALSE, family = "binomial")

# Get the best lambda value
```

```r
best_lambda <- cv_fit$lambda.min

# Fit the Lasso model with the best lambda
lasso_model <- glmnet(X_train, y_train, alpha = 1, lambda = best_lambda, standardize = FALSE, family =

# Get the coefficients
coef_lasso <- coef(lasso_model)
selected_variables <- as.data.frame(as.matrix(coef_lasso))
selected_variables <- as.data.frame(t(selected_variables))

non_zero_columns <- apply(selected_variables, 2, function(col) !all(col == 0))

# Get nonzero coefficients
non_zero_coef <- selected_variables[, non_zero_columns]

# Get the names of the non-zero features
non_zero_features <- colnames(non_zero_coef)
non_zero_features
```

```
##  [1] "(Intercept)" "REC"         "FRE"         "CC_CARD"     "PSWEATERS"
##  [6] "PKNIT_TOPS"  "PKNIT_DRES"  "PJACKETS"    "PCAS_PNTS"   "PSHIRTS"
## [11] "PDRESSES"    "PSUITS"      "POUTERWEAR"  "PJEWELRY"    "AMSPEND"
## [16] "PSSPEND"     "CCSPEND"     "AXSPEND"     "TMONSPEND"   "OMONSPEND"
## [21] "PREVPD"      "GMP"         "PROMOS"      "DAYS"        "CLASSES"
## [26] "COUPONS"     "NITEMS"      "STORES"      "VALPHON"     "WEB"
## [31] "RESPONDED"   "LTP12FRE"    "PERCRET"     "CLUSTYPE1"   "CLUSTYPE4"
## [36] "CLUSTYPE7"
```

```r
# Modify training and testing data to include only selected variables
non_zero_features <- non_zero_features[non_zero_features != "(Intercept)"]
select <- c("RESP", non_zero_features)

train_scaled <- subset(train_scaled, select = select)
train_scaled$CLUSTYPE7 <- as.factor(train_scaled$CLUSTYPE7)
test_scaled <- subset(test_scaled, select = select)
test_scaled$CLUSTYPE7 <- as.factor(test_scaled$CLUSTYPE7)
val_scaled <- subset(val_scaled, select = select)
val_scaled$CLUSTYPE7 <- as.factor(val_scaled$CLUSTYPE7)
```

Random Forest

```r
library(randomForest)
```

```
## randomForest 4.7-1.2

## Type rfNews() to see new features/changes/bug fixes.

##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
set.seed(1234)

train_control <- trainControl(method = "cv", number = 5, classProbs = TRUE, summaryFunction = twoClassSu

rf <- randomForest(RESP ~ ., data = train_scaled, family = "binomial", trControl = train_control, ntree
save(rf, file = "rf.Rdata")
predictions_val <- predict(rf, newdata = val_scaled, type = "prob")
```

Find Optimal Cutoff

```r
# Try different probability cutoffs
cutoffs <- seq(0.01, 0.99, by = 0.01)

# Calculate profit for each cutoff
profits <- sapply(cutoffs, function(cutoff) {

  pred <- ifelse(predictions_val[, 2] > cutoff, 1, 0)

  TP <- sum(pred == 1 & y_val == 1)
  FP <- sum(pred == 1 & y_val == 0)

  profit <- (170 * TP) - (30 * (TP + FP))

  return(profit)
})

# Find cutoff that maximizes profit
best_cutoff <- cutoffs[which.max(profits)]
best_profit <- max(profits)

best_cutoff
```

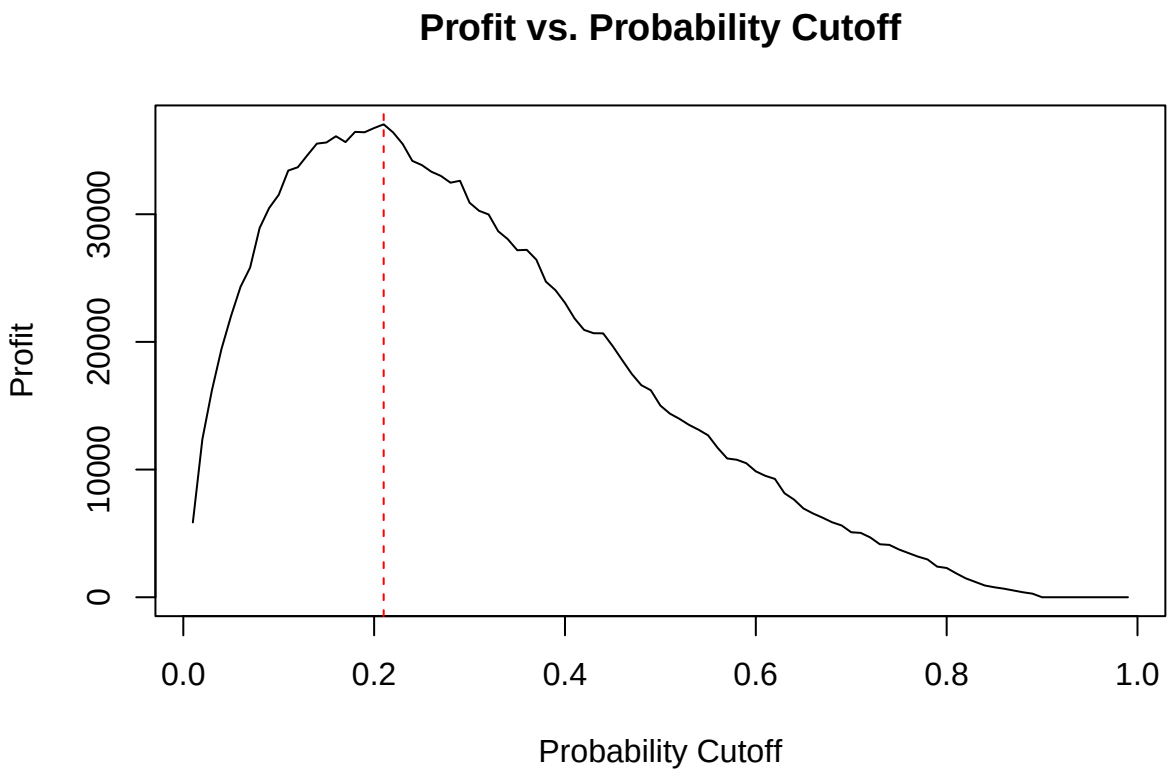
```
## [1] 0.21
```

```r
best_profit
```

```
## [1] 37040
```

```r
# Plot profit against cutoff
plot(cutoffs, profits, type = "l",
     xlab = "Probability Cutoff",
     ylab = "Profit",
     main = "Profit vs. Probability Cutoff")

abline(v = best_cutoff, col = "red", lty = 2)
```




```
cutoff <- best_cutoff
```

Prediction

```
predictions <- predict(rf, newdata = test_scaled, type = "prob")
pred <- ifelse(predictions[,2] > cutoff, 1, 0) # convert to standard predictions format, only need the
pred <- as.factor(pred) # make into a factor (for confusion matrix)

confusionMatrix(as.factor(pred), test_scaled$RESP)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1918  132
##          1  607  372
##
##                Accuracy : 0.756
##                  95% CI : (0.7403, 0.7712)
##     No Information Rate : 0.8336
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.3614
##
##  Mcnemar's Test P-Value : <2e-16
```

```
##
##             Sensitivity : 0.7596
##             Specificity : 0.7381
##          Pos Pred Value : 0.9356
##          Neg Pred Value : 0.3800
##              Prevalence : 0.8336
##          Detection Rate : 0.6332
##    Detection Prevalence : 0.6768
##       Balanced Accuracy : 0.7488
##
##        'Positive' Class : 0
##
```

```
TP <- sum(pred == 1 & y_test == 1)
FP <- sum(pred == 1 & y_test == 0)
TN <- sum(pred == 0 & y_test == 0)
FN <- sum(pred == 0 & y_test == 1)

profit_dt <- 170 * TP - 30 * (TP + FP)
print(profit_dt)
```

```
## [1] 33870
```

```
print(paste("TP:", TP, "FP:", FP, "TN:", TN, "FN:", FN))
```

```
## [1] "TP: 372 FP: 607 TN: 1918 FN: 132"
```

```
accuracy <- (TP + TN) / (TP + TN + FP + FN)

# Extra performance metrics
percent_TP_captured = TP / sum(y_test == 1) # the percent of TP we actually predict
loss_FP = 30 * FP # the amount we waste on FP
profit_TP = 140 * TP # the amount we gain from TP without the cost
max_profit = 140 * sum(y_test == 1)
potential_profit = max_profit - profit_dt

print(paste("Percent TP captured:", percent_TP_captured))
```

```
## [1] "Percent TP captured: 0.738095238095238"
```

```
print(paste("Loss from FP:", loss_FP))
```

```
## [1] "Loss from FP: 18210"
```

```
print(paste("Profit from TP:",profit_TP))
```

```
## [1] "Profit from TP: 52080"
```

```r
print(paste("Potential Profit Left:",potential_profit))
```

```
## [1] "Potential Profit Left: 36690"
```

```r
print(paste("Accuracy:", accuracy))
```

```
## [1] "Accuracy: 0.756025090789039"
```

```r
# 1. Base Financials
gross_revenue_per_responder <- 170
cost_per_brochure           <- 30
net_profit_per_responder    <- gross_revenue_per_responder - cost_per_brochure # $140

# 2. Campaign Performance Metrics
# Total Profit = (Gross Revenue from Buyers) - (Total Cost of Mailing Everyone)
profit_dt <- (170 * TP) - (30 * (TP + FP))

accuracy  <- (TP + TN) / (TP + TN + FP + FN)
recall    <- TP / sum(y_test == 1)  # Your percent_TP_captured variable

# 3. Cost & Opportunity Breakdown
loss_FP          <- cost_per_brochure * FP              # Pure waste on non-responders
revenue_gained   <- net_profit_per_responder * TP       # Net money made from your targeted buyers
max_profit       <- net_profit_per_responder * sum(y_test == 1) # If you had a perfect crystal ball

# Money left on the table (Lost profit from False Negatives you didn't mail)
money_left_on_table <- max_profit - profit_dt

# Percent of profit captured
percent_profit <- round(100*(profit_dt)/max_profit,2)

# --- Print Summary ---
print(paste("Confusion Matrix -> TP:", TP, "| FP:", FP, "| TN:", TN, "| FN:", FN))
```

```
## [1] "Confusion Matrix -> TP: 372 | FP: 607 | TN: 1918 | FN: 132"
```

```r
print(paste("Model Accuracy:      ", round(accuracy * 100, 2), "%"))
```

```
## [1] "Model Accuracy:       75.6 %"
```

```r
print(paste("Responders Captured: ", round(recall * 100, 2), "%"))
```

```
## [1] "Responders Captured:  73.81 %"
```

```r
print("-------------------------------------------------")
```

```
## [1] "-------------------------------------------------"
```

```r
print(paste("Actual Net Profit:   $", profit_dt))
```

```
## [1] "Actual Net Profit:   $ 33870"
```

```r
print(paste("Wasted Mailing Cost: $", loss_FP))
```

```
## [1] "Wasted Mailing Cost: $ 18210"
```

```r
print(paste("Max Possible Profit: $", max_profit))
```

```
## [1] "Max Possible Profit: $ 70560"
```

```r
print(paste("Percent profit captured:", percent_profit, "%"))
```

```
## [1] "Percent profit captured: 48 %"
```

```r
print(paste("Money Left on Table: $", money_left_on_table))
```

```
## [1] "Money Left on Table: $ 36690"
```

Logistic Baseline

```r
# Logistic
logistic <- glm(RESP ~. , data = train_scaled, family = binomial)
predictions_val <- predict(logistic, newdata = val_scaled, type = "response")

# Try different probability cutoffs
cutoffs <- seq(0.01, 0.99, by = 0.01)

# Calculate profit for each cutoff
profits <- sapply(cutoffs, function(cutoff) {

  pred <- ifelse(predictions_val > cutoff, 1, 0)

  TP <- sum(pred == 1 & y_val == 1)
  FP <- sum(pred == 1 & y_val == 0)

  profit <- (170 * TP) - (30 * (TP + FP))

  return(profit)
})

# Find cutoff that maximizes profit
best_cutoff <- cutoffs[which.max(profits)]
best_profit <- max(profits)

cutoff <- best_cutoff

predictions <- predict(logistic, newdata = test_scaled, type = "response")
predicted_label <- ifelse(predictions > cutoff, "1", "0")
confusionMatrix(as.factor(predicted_label), test_scaled$RESP)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1877  126
##          1  648  378
##
##                Accuracy : 0.7445
##                  95% CI : (0.7285, 0.7599)
##     No Information Rate : 0.8336
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.3488
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.7434
##             Specificity : 0.7500
##          Pos Pred Value : 0.9371
##          Neg Pred Value : 0.3684
##              Prevalence : 0.8336
##          Detection Rate : 0.6197
##    Detection Prevalence : 0.6613
##       Balanced Accuracy : 0.7467
##
##        'Positive' Class : 0
##
```

```
TP <- sum(predicted_label == 1 & test_data$RESP == 1)
FP <- sum(predicted_label == 1 & test_data$RESP == 0)
log_profit <- 170 * TP - 30 * (TP + FP)
log_profit
```

```
## [1] 33480
```

```
# Percent of profit captured
percent_profit <- round(100*(log_profit)/max_profit,2)
percent_profit
```

```
## [1] 47.45
```